Использование библиотеки NumPy в матричных вычислениях

Матричные вычисления находят широкое применение в различных областях науки, техники и бизнеса. Они применяются в различных областях, таких, как:

1. **Машинное обучение и искусственный интеллект**:
    - Матричные операции являются основой для многих алгоритмов машинного обучения, включая линейную регрессию, нейронные сети и методы кластеризации.
    - Векторизация данных и использование матриц позволяют ускорить вычисления и эффективно обрабатывать большие объемы информации.
2. **Компьютерная графика**:
    - Матричные вычисления используются для трансформации объектов в 2D и 3D пространстве, включая повороты, масштабирование и перемещение.
    - Применяются в рендеринге, анимации и моделировании.
3. **Обработка сигналов и изображений**:
    - В обработке изображений матричные операции используются для фильтрации, преобразования и анализа изображений.
    - В обработке сигналов матрицы могут представлять временные ряды и использоваться для анализа и фильтрации сигналов.
4. **Физика и инженерия**:
    - Матричные вычисления применяются в механике, электротехнике и других областях для решения систем линейных уравнений, моделирования динамических систем и анализа структур.
5. **Экономика и финансы**:
    - В экономических моделях и финансовом анализе матричные вычисления используются для обработки данных, оптимизации портфелей и анализа рисков.
6. **Научные вычисления**:
    - В научных исследованиях матричные вычисления применяются для моделирования и симуляции различных процессов, включая биологические, химические и физические системы.
7. **Системы управления**:
    - В теории управления матричные методы используются для анализа и проектирования систем управления, включая управление динамическими системами.
8. **Криптография**:

- Некоторые криптографические алгоритмы используют матричные операции для шифрования и дешифрования данных.

В данном разделе вы узнаете:

1. Как создавать различные виды массивов (единичные, состоящие из нулей, состоящие из единиц);
2. Как осуществлять операции над массивами;
3. Находить определитель матрицы;
4. Находить обратную матрицу.

Создание матриц (массивов) в NumPy

Ранее мы рассмотрели, как создавать массив NumPy из списков Python, из файлов. Кроме рассмотренных способов, в NumPy есть дополнительные способы создания массивов:

1. Создание единичной матрицы.

```
b = np.eye(5)
print ("Единичная матрица:\n", b)
```

Выражение b = np.eye(5) создает единичную матрицу размером 5x5 (в скобках задается размер единичной матрицы). Единичная матрица — это квадратная матрица, в которой все элементы на главной диагонали равны 1, а все остальные элементы равны 0. Результатом выполнения кода будет:

```
Единичная матрица:
 [[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
```

2. Создание массива (матрицы), состоящей из единиц:

```
c = np.ones((7, 5))
print ("Матрица, состоящая из одних единиц:\n", c)
```

выражение c = np.ones((7, 5)) создает двумерный массив (матрицу) размером 7x5, заполненный единицами.

Объяснение:

- np.ones((7, 5)) — это функция из библиотеки NumPy, которая генерирует массив заданной формы, заполненный значениями 1.

- Аргумент (7, 5) указывает размерность создаваемого массива: 7 строк и 5 столбцов.

Результатом выполнения кода будет:

Матрица, состоящая из одних единиц:

```
[[1. 1. 1. 1. 1.]
[1. 1. 1. 1. 1.]
[1. 1. 1. 1. 1.]
[1. 1. 1. 1. 1.]
[1. 1. 1. 1. 1.]
[1. 1. 1. 1. 1.]
[1. 1. 1. 1. 1.]]
```

3. Создание матрицы заданного размера, состоящей из заданного значения:

```
d = np.full((2,2), 7)  # Создает матрицу (1, 2) заполненую заданным значением
print(d)
```

выражение d = np.full((2, 2), 7) создает двумерный массив (матрицу) размером 2x2, заполненный значением 7.

Объяснение:

- np.full((2, 2), 7) — это функция из библиотеки NumPy, которая генерирует массив заданной формы, заполненный указанным значением.
- Аргумент (2, 2) указывает размерность создаваемого массива: 2 строки и 2 столбца.
- Второй аргумент 7 указывает, что все элементы массива должны быть равны 7.

```
[[ 7.  7.]
 [ 7.  7.]]
```

4. Создание матрицы, заполненной случайными цифрами:

```
e = np.random.random((2,2))
print(e)
```

выражение e = np.random.random((2, 2)) создает двумерный массив (матрицу) размером 2x2, заполненный случайными числами, равномерно распределенными в диапазоне от 0 до 1.

Объяснение:

- np.random.random((2, 2)) — это функция из модуля random библиотеки NumPy, которая генерирует массив заданной формы, заполненный случайными числами.
- Аргумент (2, 2) указывает размерность создаваемого массива: 2 строки и 2 столбца.

В результате выполнения кода получим:

```
[[0.25744383 0.48056466]
 [0.13767881 0.40578168]]
```

Задание

1. Создайте единичную матрицу с количеством столбцов, равным 4.
2. Создайте матрицу, состоящую из 1 размером 2x3.
3. Выведите результаты в консоль.

Векторы. Вектор-строки и вектор-столбцы

В NumPy векторы могут быть представлены как векторы-строки (1xN) и векторы-столбцы (Nx1). Различие между ними заключается в их форме (shape) и в том, как они хранятся в памяти.

Вектор-строка — это двумерный массив с одной строкой и несколькими столбцами. Его форма будет (1, N).

Пример создания вектора-строки:

```
import numpy as np
# Создаем вектор-строку
row_vector = np.array([[1, 2, 3]])  # Обратите внимание на двойные квадратные скобки
print("Вектор-строка:")
print(row_vector)
print ("Форма:", row_vector.shape)
```

В результате будет выведено:

```
Вектор-строка:
[[1 2 3]]
Форма: (1, 3)
```

Вектор-столбец — это двумерный массив с несколькими строками и одним столбцом. Его форма будет (N, 1).

Пример создания вектора-столбца:

```
# Создаем вектор-столбец
column_vector = np.array([[1], [2], [3]])  # Обратите внимание на двойные
квадратные скобки

print("Вектор-столбец:")
print(column_vector)
print("Форма:", column_vector.shape)
```

В результате будет выведено в консоль:

```
Вектор-столбец:

[[1]

 [2]

 [3]]

Форма: (3, 1)
```

Вы также можете использовать методы reshape для изменения формы векторов:

```
# Создаем одномерный массив
array = np.array([1, 2, 3])

# Преобразуем вектор в вектор-строку
row_vector = array.reshape(1, -1)  # -1 позволяет NumPy автоматически
определить размер
print("Вектор-строка (reshape):")
print(row_vector)

# Преобразуем вектор в вектор-столбец
column_vector = array.reshape(-1, 1)
print("Вектор-столбец (reshape):")
print(column_vector)
```

В результате будет выведено в консоль:

```
Вектор-строка (reshape):

[[1 2 3]]

Вектор-столбец (reshape):

[[1]

 [2]

 [3]]
```

reshape(1, -1) — это метод, который изменяет форму массива.

- 1 указывает, что вы хотите, чтобы в новом массиве была 1 строка.
- -1 — это специальный параметр, который говорит NumPy автоматически определить размер этого измерения на основе общего количества элементов в массиве. В данном случае это будет количество столбцов.

Математические операции над матрицами

К массивам (матрицам) можно применять известные вам математические операции. Следует понимать, что при этом у элементов должны быть схожие размерности.

Рассмотрим на примере. Предположим, у нас есть следующие матрицы, заданные массивами NumPy:

```
x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)
arr = np.array([1, 2])
```

dtype=np.float64 указывает, что элементы массива должны быть типа float64, что означает 64-битное число с плавающей запятой. Это важно, если вы хотите, чтобы все значения в массиве были представлены как числа с плавающей запятой, а не как целые числа.

Для начала рассмотри способы поэлементного сложения. Существует несколько способов сложения массивов одинаковой размерности:

```
print(x + y)
print()
print(np.add(x, y))
```

Оператор + и функция np.add работают аналогично и позволяют прибавлять массивы к массивам и числа к массивам.

В результате выполнения этого кода будет выведено:

```
[[ 6.  8.]
 [10. 12.]]

[[ 6.  8.]
 [10. 12.]]
```

В результате операции сложения было выполнено сложения элементов, имеющих одинаковые индексы.

Операцию сложения можно выполнить так же с числом и массивом другой размерности. Рассмотрим на примере.

Попробуем сложить массив x и arr.

```
print('С массивом другой размерности')
print(x + arr)
```

В результате выполнения данного кода получим:

С массивом другой размерности

```
[[2. 4.]
 [4. 6.]]
```

Как видно из получившегося результата, массив arr был поэлементно добавлен к каждой строке массива x. Однако следует помнить, что количество элементов строке массива должно быть меньше либо равно количеству элементов второго массива, инвче будет выведена ошибка.

Сложение массива с числом обсуждалось ранее в предыдущей лабораторной работе. Приведем пример, но более подробно обсуждать не будем.

```
print('С числом')
print(x + 1)
```

Результат:

```
[[2. 3.]
```

```
[4. 5.]]
```

Вычитание и деление выполняется аналогично.

Пример вычитания:

```
# Вычитание
print(x - y)
print(np.subtract(x, y))
```

В результате будет выведено:

```
[[-4. -4.]
 [-4. -4.]]
[[-4. -4.]
 [-4. -4.]]
```

Как и в случае сложения, было выполнено поэлементное вычитание.

Вычитание также можно выполнять с массивом другой размерности и числами.

Аналогично выполняется операция деления:

```
print(x / y)
print(np.divide(x, y))
```

В результате будет выведено:

```
[[0.2        0.33333333]
 [0.42857143 0.5       ]]
[[0.2        0.33333333]
 [0.42857143 0.5       ]]
```

При делении не следует забывать про то, что в python не определена операция деления на 0.

С умножением сложнее. Существует два способа умножения: матричное и поэлементное. Поэтому рассмотрим отдельным разделом.

Матричное и поэлементное умножение

Матричное умножение — это операция, которая принимает две матрицы и производит новую матрицу, элементы которой являются результатом линейных комбинаций строк одной матрицы и столбцов другой. Эта операция широко используется в линейной алгебре, статистике, машинном обучении и многих других областях.

Условия для матричного умножения

Чтобы две матрицы можно было перемножить, количество столбцов первой матрицы должно быть равно количеству строк второй матрицы. Если у нас есть матрица A размером

m×n и матрица B размером n×p, то результатом их умножения будет матрица C размером m×p.

Определение

Если A — это матрица размером m×n, а B — матрица размером n×p, то элемент c_{ij} матрицы C (результат умножения) вычисляется по формуле:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

где:

- c_{ij}— элемент в строке i и столбце j матрицы C,
- a_{ik} — элемент в строке i и столбце k матрицы A,
- b_{kj} — элемент в строке k и столбце j матрицы B.

В NumPy существует несколько способов выполнения матричного умножения.

1. Использование оператора @

С помощью оператора @ можно выполнять матричное умножение. Этот оператор был введен в Python 3.5 и является наиболее удобным способом для выполнения матричного умножения.

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

C = A @ B
print(C)
```

C1 (using @):

```
[[19 22]
 [43 50]]
```

Если использовать его с одномерными массивами (векторами), он будет рассчитывать скалярное произведение.

2. Использование функции np.dot()

Функция np.dot() также может использоваться для выполнения матричного умножения. Она работает как для двумерных массивов (матриц), так и для одномерных (векторов).

```
C = np.dot(A, B)
print(C)
```

В результате получим:

```
C2 (using np.dot()):

[[19 22]

 [43 50]]
```

Если оба аргумента — двумерные массивы, то np.dot() выполняет матричное умножение. Если один из аргументов — одномерный массив, то он выполняет скалярное произведение.

3. Использование функции np.matmul()

Функция np.matmul() предназначена специально для матричного умножения. Она работает аналогично оператору @.

```
C = np.matmul(A, B)
print(C)
```

В результате выполнения этого кода получим тот же результат:

```
C3 (using np.matmul()):

[[19 22]

 [43 50]]
```

np.matmul() была введена в NumPy 1.10 и предназначена исключительно для матричного умножения. Работает аналогично оператору @, но не поддерживает скалярное произведение. Если вы передаете одномерные массивы, вы получите ошибку.

4. Использование метода .dot() для объектов массива

Вы также можете использовать метод .dot() для объектов массива NumPy.

```
C = A.dot(B)
print(C)
```

В результате выполнения этого кода получим тот же результат:

Работает аналогично np.dot(), но также может быть использован для выполнения матричного умножения. Если вы передаете одномерные массивы, он будет вести себя как скалярное произведение.

Обратите внимание: операция * производит над матрицами покоординатное умножение, а не матричное!

Рассмотрим пример:

```
r = a * b
print ("Матрица A:\n", a)
print ("Матрица B:\n", b)
print ("Результат покоординатного умножения через операцию умножения:\n", r)
```

В консоль будет выведено:

Матрица A:

```
[[1 0]
 [0 1]]
Матрица B:
[[4 1]
 [2 2]]
Результат покоординатного умножения через операцию умножения:
[[4 0]
 [0 2]]
```

Изменение формы массива

В библиотеке NumPy существует возможность изменять формы массива различными способами. Начнем с добавления столбцов и строк путем объединения массивов.

Функция np.vstack() из библиотеки NumPy используется для вертикального объединения (стacking) массивов. Она принимает кортеж или список массивов и объединяет их по вертикали, то есть добавляет строки одного массива к строкам другого массива.

Синтаксис:

```
numpy.vstack(tup)
```

tup: Кортеж или список массивов, которые вы хотите объединить. Все массивы должны иметь одинаковую ширину (то есть одинаковое количество столбцов).

Рассмотрим пример:

```
import numpy as np

# Создание двух массивов
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

# Вертикальное объединение массивов
c = np.vstack((a, b))

print("Массив a:")
print(a)

print("\nМассив b:")
print(b)

print("\nОбъединенный массив c:")
print(c)
```

В результате выполнения данного кода получим:

Массив a:

```
[[1 2]
 [3 4]]

Массив b:
[[5 6]
 [7 8]]
Объединенный массив c:
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

В этом примере массив a имеет размерность 2×2 (2 строки и 2 столбца), и массив b также имеет размерность 2×2.

После применения np.vstack((a, b)) мы получаем новый массив c, который имеет размерность 4×2 (4 строки и 2 столбца), где строки массива b добавлены под строками массива a.

Функция np.hstack() из библиотеки NumPy используется для горизонтального объединения (stacking) массивов. Она принимает кортеж или список массивов и объединяет их по горизонтали, то есть добавляет столбцы одного массива к столбцам другого массива.

Синтаксис:

```
numpy.hstack(tup)
```

tup: Кортеж или список массивов, которые вы хотите объединить. Все массивы должны иметь одинаковую высоту (то есть одинаковое количество строк).

Рассмотрим пример:

```
import numpy as np
# Создание двух массивов
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

# Горизонтальное объединение массивов
c = np.hstack((a, b))

print("Массив a:")
print(a)

print("\nМассив b:")
print(b)

print("\nОбъединенный массив c:")
print(c)
```

Выполнив данный код, мы получим следующий результат:

```
Массив a:
[[1 2]
 [3 4]]

Массив b:
[[5 6]
 [7 8]]

Объединенный массив c:
[[1 2 5 6]
 [3 4 7 8]]
```

В этом примере массив a имеет размерность 2×2 (2 строки и 2 столбца), и массив b также имеет размерность 2×2.

После применения np.hstack((a, b)) мы получаем новый массив c, который имеет размерность 2×4 (2 строки и 4 столбца), где столбцы массива b добавлены справа от столбцов массива a.

Для изменения размерности массива так же можно использовать функцию изменения формы reshape.

Функция reshape() в NumPy используется для изменения формы (размерности) массива без изменения его данных. Метод reshape() позволяет вам указать новую форму массива, которая должна соответствовать общему количеству элементов в исходном массиве.

Синтаксис:

```
numpy.ndarray.reshape(newshape)
```

newshape: Кортеж или список, определяющий новую форму массива. Размеры в newshape должны соответствовать общему количеству элементов в исходном массиве.

Рассмотрим пример:

```
import numpy as np

# Создание одномерного массива
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Изменение формы массива на (5, 2)
b = a.reshape((5, 2))

print("Исходный массив a:")
print(a)

print("\nИзмененный массив b:")
print(b)
```

При выполнении этого кода вы получите следующий вывод:

```
Исходный массив a:
[ 1  2  3  4  5  6  7  8  9 10]


Измененный массив b:
[[ 1  2]
 [ 3  4]
 [ 5  6]
 [ 7  8]
 [ 9 10]]
```

В этом примере мы создали одномерный массив a с 10 элементами.

Затем мы использовали a.reshape((5, 2)), чтобы изменить форму массива на 5×2 (5 строк и 2 столбца).

Новый массив b теперь имеет 5 строк и 2 столбца, и все элементы из исходного массива a были распределены по новой форме.

Общее количество элементов в новом массиве должно совпадать с количеством элементов в исходном массиве. В нашем примере 10 элементов в одномерном массиве a соответствуют 10 элементам в двумерном массиве b (5 строк по 2 элемента).

Вы можете использовать -1 в качестве одного из размеров в newshape, чтобы NumPy автоматически вычислил этот размер.

```
b = a.reshape((5, -1))  # NumPy автоматически определит количество столбцов
```

Кроме того, в NumPy возможно преобразовывать двумерный массив в одномерный.

Метод flatten() в NumPy используется для преобразования многомерного массива в одномерный. Он возвращает копию массива, в которой все элементы расположены в одномерной последовательности.

Синтаксис:

```
numpy.ndarray.flatten(order='C')
```

order: Опциональный параметр, который определяет порядок, в котором элементы будут располагаться в одномерном массиве. Возможные значения:

- 'C': По умолчанию, элементы располагаются в порядке "C" (строчно, т.е. по строкам).
- 'F': Элементы располагаются в порядке "Fortran" (столбцово, т.е. по столбцам).
- 'A': Если массив является "C"-порядка, используется порядок "C", если "F"-порядка — используется порядок "F".

Рассмотрим пример:

```
import numpy as np

# Создание двумерного массива
a = np.array([[1, 2, 3], [4, 5, 6]])

# Преобразование в одномерный массив
b = a.flatten()

print("Исходный массив a:")
print(a)

print("\nПреобразованный одномерный массив b:")
print(b)
```

При выполнении этого кода вы получите следующий вывод:

```
Исходный массив a:
[[1 2 3]
 [4 5 6]]

Преобразованный одномерный массив b:
[1 2 3 4 5 6]
```

В этом примере мы создали двумерный массив a размером 2×3.

Затем мы использовали a.flatten(), чтобы преобразовать его в одномерный массив b, который содержит все элементы массива a в одномерной последовательности.

Метод flatten() возвращает копию массива. Если вы измените элементы в одномерном массиве, это не повлияет на исходный многомерный массив.

Задание

1. Создайте два одномерных массива a и b размером 10, заполненных случайными числами от 0 до 10.
2. Выполните следующие операции:
   - Сложите массивы a и b.
   - Вычтите массив b из массива a.
   - Разделите массив a на массив b (учтите, что элементы массива b не должны быть равны нулю).
3. Выведите результаты каждой операции.
4. Создайте две матрицы `A` и `B` размером 3x3, заполненные случайными числами от 1 до 10.
5. Выполните матричное умножение матриц `A` и `B`.
6. Выведите результат матричного умножения.
7. Создайте одномерный массив `C` размером 12, заполненный случайными числами от 1 до 100.
8. Измените форму массива `C` на 3x4.
9. Выведите новый массив и его форму.
10. Измените форму массива `C` на 4x3 и снова выведите его.

Транспонирование матриц

Транспонирование матриц — это операция, при которой строки матрицы становятся столбцами, а столбцы — строками. В NumPy это можно сделать с помощью метода .T или функции numpy.transpose().

Например,

```
import numpy as np

# Создание матрицы 2x3
A = np.array([[1, 2, 3],
              [4, 5, 6]])
print("Исходная матрица A:")
print(A)

# Транспонирование матрицы с помощью метода .T
A_transposed = A.T
print("\nТранспонированная матрица A (метод .T):")
print(A_transposed)

# Транспонирование матрицы с помощью функции numpy.transpose()
A_transposed_func = np.transpose(A)
print("\nТранспонированная матрица A (функция numpy.transpose):")
print(A_transposed_func)
```

При выполнении этого кода вы получите следующий вывод:

```
Исходная матрица A:
[[1 2 3]
 [4 5 6]]

Транспонированная матрица A (метод .T):
[[1 4]
 [2 5]
 [3 6]]

Транспонированная матрица A (функция numpy.transpose):
[[1 4]
 [2 5]
 [3 6]]
```

Транспонирование не изменяет исходный массив, а создает новый массив с измененной формой.

Определитель матрицы

Определитель матрицы — это скалярная величина, которая может быть вычислена для квадратной матрицы (матрицы с одинаковым числом строк и столбцов).

Для вычисления определителя матрицы в NumPy можно использовать функцию numpy.linalg.det(). Вот пример:

```
import numpy as np

# Создание квадратной матрицы 2x2
A = np.array([[4, 2],
              [3, 1]])

# Вычисление определителя
det_A = np.linalg.det(A)

print("Определитель матрицы A:")
print(det_A)
```

```
# Создание квадратной матрицы 3x3
B = np.array([[1, 2, 3],
              [0, 1, 4],
              [5, 6, 0]])

# Вычисление определителя
det_B = np.linalg.det(B)

print("Определитель матрицы B:")
print(det_B)
```

При выполнении этих кодов вы получите определители для заданных матриц. Например, для матрицы A определитель будет равен -2, а для матрицы B — -14.

Ранг матрицы

Ранг матрицы — это важное понятие в линейной алгебре, которое определяет максимальное количество линейно независимых строк или столбцов в матрице. Ранг матрицы дает информацию о ее структуре и свойствах, и он может быть использован для решения различных задач, таких как:

1. **Определение линейной зависимости**: Если ранг матрицы равен числу строк (или столбцов), то все строки (или столбцы) линейно независимы. Если ранг меньше, то некоторые строки (или столбцы) могут быть выражены через другие.
2. **Решение систем линейных уравнений**: Ранг матрицы помогает определить, имеет ли система линейных уравнений решение, и если да, то сколько решений она имеет.

В NumPy для вычисления ранга матрицы можно использовать функцию numpy.linalg.matrix_rank(). Вот пример:

```
import numpy as np

# Создание матрицы
A = np.array([[1, 2, 3],
```

```
              [4, 5, 6],
              [7, 8, 9]])

# Вычисление ранга матрицы
rank_A = np.linalg.matrix_rank(A)

print("Ранг матрицы A:")
print(rank_A)
# Создание другой матрицы
B = np.array([[1, 2, 3],
              [0, 1, 4],
              [5, 6, 0]])

# Вычисление ранга матрицы
rank_B = np.linalg.matrix_rank(B)

print("Ранг матрицы B:")
print(rank_B)
```

При выполнении этих кодов вы получите ранги для заданных матриц. Например, для матрицы A ранг будет равен 2, так как строки линейно зависимы, а для матрицы B ранг будет равен 3, так как все строки линейно независимы.

Обратные матрицы

Обратная матрица — это матрица, которая при умножении на исходную матрицу дает единичную матрицу. Обратная матрица обозначается как A^{-1}, где A — исходная матрица. Обратная матрица существует только для квадратных матриц (матриц с одинаковым числом строк и столбцов), и только если определитель этой матрицы не равен нулю.

В NumPy для вычисления обратной матрицы можно использовать функцию numpy.linalg.inv(). Например:

```
import numpy as np

# Создание квадратной матрицы
A = np.array([[4, 7],
              [2, 6]])

# Вычисление обратной матрицы
A_inv = np.linalg.inv(A)

print("Исходная матрица A:")
print(A)

print("\nОбратная матрица A^-1:")
print(A_inv)

# Проверка: умножение матрицы на её обратную
identity_matrix = np.dot(A, A_inv)
print("\nПроверка: A * A^-1 =")
print(identity_matrix)
```

При выполнении этого кода вы получите обратную матрицу для заданной матрицы A и проверку, что произведение матрицы на её обратную дает единичную матрицу.

```
Исходная матрица A:
```

```
[[4 7]
 [2 6]]

Обратная матрица A^-1:
[[ 0.6 -0.7]
 [-0.2  0.4]]

Проверка: A * A^-1 =
[[ 1.00000000e+00  0.00000000e+00]
 [-2.22044605e-16  1.00000000e+00]]
```

Задание

1. Создайте квадратную матрицу A размером 4x4, заполненную случайными целыми числами от 1 до 10.
2. Выполните транспонирование матрицы A.
3. Выведите исходную матрицу и её транспонированную версию.
4. Создайте матрицу `C` размером 5x3, заполненную случайными целыми числами от 1 до 10.
5. Вычислите ранг матрицы `C` и её транспонированной версии `C.T`.
6. Сравните ранги матрицы `C` и её транспонированной версии. Что вы можете сказать о них?
7. Создайте матрицу `D` размером 4x5, значения которой должны быть целыми числами на отрезке от 1 до 5, представляющую данные о студентах (оценки по 5 предметам).
8. Транспонируйте матрицу `D`, чтобы получить матрицу, где строки будут представлять предметы, а столбцы — студентов.
9. Вычислите ранг матрицы `D` и её транспонированной версии. Какой вывод можно сделать о линейной зависимости оценок?

Итоги

Таким образом, в рамках данной работы вы узнали:

1. Как выполнять операции над массивами;
2. Как выполнять матричное умножение;
3. Находить ранг и определитель матрицы;
4. Находить обратную матрицу.

Задание

1. В рамках данного задания необходимо выполнить обработку изображения. Для загрузки изображения вам понадобится библиотека pillow.

Pillow — это библиотека для работы с изображениями в Python. Она является улучшенной версией библиотеки PIL (Python Imaging Library) и предоставляет множество функций для открытия, манипуляции и сохранения различных форматов изображений.

Кроме того, для просмотра результата импортируем библиотеку визуализации matplotlib.

Библиотеки pillow и matplotlib необходимо установить перед их использованием.

```
pip install Pillow
```

и

```
pip install matplotlib
```

Далее выполним все необходимые импорты:

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
```

Загрузим исходное изображение:

```
image = Image.open('cat_PNG50550.png')
```

2. Преобразуйте image в массив NumPy с помощью функции array.
3. Далее преобразуем изображение в черно-белое. Для этого необходимо исходное изображение умножить на вектор [0.299, 0.587, 0.114]. При преобразовании цветного изображения в черно-белое (или градации серого) используются веса для цветовых каналов (красный, зеленый и синий), чтобы учесть различное восприятие яркости человеческим глазом. Значения [0.299, 0.587, 0.114] представляют собой стандартные коэффициенты, которые отражают относительную важность каждого цветового канала при вычислении яркости.

Выведите форму массива, содержащего изображение. Исходное изображение имеет следующую форму массива:

(528, 557, 4)

Форма массива изображения (528, 557, 4) указывает на то, что изображение имеет следующие характеристики:

- 528: это высота изображения в пикселях. То есть изображение состоит из 528 строк пикселей.
- 557: это ширина изображения в пикселях. То есть изображение состоит из 557 столбцов пикселей.

- 4: это количество цветовых каналов. В данном случае, значение 4 обычно указывает на то, что изображение имеет четыре канала: красный (R), зеленый (G), синий (B) и альфа (A). Альфа-канал отвечает за прозрачность пикселей.

1. Для изменения цветов нам необходимо умножить цветовые каналы на указанный выше вектор, для этого нужно сделать срез нашего изображения таким образом, чтобы не изменить длину и ширину. Для этого используем image_array[..., :3]. . . . **(троеточие) -** это специальный синтаксис в NumPy, который позволяет указать, что мы хотим сохранить все предыдущие измерения массива. В данном случае это означает, что мы хотим сохранить все значения по первым двум измерениям (высота и ширина).

   :3 — это срез, который выбирает первые три канала (0, 1 и 2) из последнего измерения массива. В контексте RGB это означает, что мы выбираем все пиксели, но только для трех цветовых каналов (красный, зеленый и синий).

   Выполните скалярное умножение image_array[..., :3] и вектора [0.299, 0.587, 0.114].

4. Сохраните полученное изображение в переменную gray_image, преобразовав его из массива NumPy в изображение с помощью метода Image.fromarray(gray_image_array) библиотеки Pillow.
5. Теперь изменим размер черно-белого изображения. Задайте новый размер в виде кортежа со значениями длины и ширины, равными 200, и запишите в переменную new_size.
6. Измените размер изображения с помощью функции resize:

```
resized_image = gray_image.resize(new_size)
```

7. Сохраните полученное изображение с помощью функции save()

```
resized_image.save('processed_image.jpg')
```

8. Далее выведем полученные изображения с помощью библиотеки matplotlib. Для этого используем объект plt.imshow. plt.imshow — это функция из библиотеки Matplotlib, которая используется для отображения изображений в графическом интерфейсе. Она позволяет визуализировать двумерные массивы, такие как изображения, в виде графиков.

Более подробно о данной библиотеке вы узнаете позже. Сейчас скопируйте код ниже и вставьте к себе.

```
plt.figure(figsize=(10, 5))

plt.subplot(1, 3, 1)
plt.title('Исходное изображение')
plt.imshow(image)
plt.axis('off')

plt.subplot(1, 3, 2)
plt.title('Оттенки серого')
plt.imshow(gray_image, cmap='gray')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.title('Измененный размер')
plt.imshow(resized_image, cmap='gray')
```

```
plt.axis('off')

plt.tight_layout()
plt.show()
```

9. Возьмите свое похожее изображение и повторите все шаги.